

“INTERNSHIP REPORT”

At The DRDO (Defence Research and Development Organization)

Titled

“Windows Endpoint SIEM and SOC Console”

Submitted By

Y Nishanth Reddy

Roll No: 23315A6207

Sreenidhi Institute of Science and Technology

Hyderabad, Telangana, India

October 2026



BONAFIDE CERTIFICATE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my internship mentors, Shri **PRAMOD KUMAR JHA SCIENTIST- 'G'** sir and **SMT B. BEULAH ASWINI** Ma'am [TO- 'C'], **DRDO, Hyderabad** for their constant guidance, encouragement, and valuable insights throughout my internship. Their technical expertise, constructive feedback, and support provided me with numerous opportunities to learn and apply cybersecurity concepts in practical environments. I am also thankful to my fellow colleagues at the organization for their cooperation, knowledge sharing, and friendly work environment, which made my internship a valuable learning experience.

I would also like to extend my heartfelt thanks to the **Sreenidhi Institute of Science and Technology(SNIST)**, along with the faculty members of the Cyber Security, for providing me with the academic foundation, continuous guidance, and the opportunity to undertake this internship as part of my postgraduate curriculum. The knowledge and skills gained during my coursework greatly contributed to the successful completion of my internship. Finally, I am deeply grateful to my family and friends for their unwavering encouragement, patience, and support throughout this journey. Their motivation and belief in me helped me overcome challenges and successfully complete this internship. I sincerely thank everyone who contributed, directly or indirectly, to making this experience both enriching and rewarding.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO.
1	Background 1.1 Introduction 1.2 Brief of the Internship	5
2	Objectives of the Internship 2.1 Internship Objectives	6
3	About the Organisation	6
4	Role at the organization	7
5	Learnings & Experiences 5.1 Project Overview and Architecture 5.2 Windows Event Collection 5.3 Detection and Correlation Engine 5.4 Endpoint Monitoring 5.5 Quantitative Evaluation of the Developed SIEM 5.6 SOC Dashboard and Web Application	8-29
6	Conclusion	30
7	Appendix	31-32
8	Citations	33
9	References	34

1.BACKGROUND

1.1 Introduction

Cybersecurity monitoring is essential for organizations that depend on Windows endpoints because operating systems continuously generate event records, system changes, application failures, device activity and network evidence. Security teams need a practical way to collect this information, store it in a structured form and identify events that deserve investigation.

The Windows Endpoint SIEM and SOC Console supplied for this internship is a Windows-focused security monitoring application developed with Python, Flask and SQLite. It brings together selected Windows Event Viewer records and endpoint information and presents them through a browser-based Security Operations Center interface. The project is designed as an educational local SIEM rather than as a replacement for a managed enterprise SIEM or a full EDR product.

The supplied project archive contains collectors, detection and correlation logic, endpoint-monitoring modules, a SQLite database, a Flask web interface, reporting utilities and automated tests. The populated database provides actual runtime evidence that the implementation has processed structured security records rather than containing only demonstration screens.

1.2 Brief of the Internship

The internship activity focused on understanding and evaluating the supplied Windows Endpoint SIEM project from both implementation and operational perspectives. The work involved studying the project architecture, examining the Python modules responsible for collection and detection, inspecting the SQLite data model, reviewing the Flask dashboard and analysing the stored runtime records.

The project collects selected Windows Security, System and Application events and combines them with endpoint observations such as installed software, browser information, Windows updates, USB devices, hardware and drivers, firewall state, listening ports and packet-rate measurements. The collected information is stored in SQLite, processed by rule-based detection logic and, where applicable, combined by a correlation engine to produce higher-level findings.

The internship also involved reviewing the supplied automated tests and documenting implementation limits. The analysis confirmed that the current system is rule-based, that the supplied database contains System and Application logs but no Security-channel records, and that several endpoint checks can legitimately return UNKNOWN values.

2. OBJECTIVES OF THE INTERNSHIP

2.1 Internship Objectives

The internship was undertaken with the primary objective of gaining practical experience in Windows endpoint security monitoring, SIEM architecture, event analysis and cybersecurity software development. The work emphasized understanding how endpoint evidence can be collected, normalized, detected, correlated and presented through a standalone security console.

1. To study the architecture and working principles of the supplied Windows Endpoint SIEM and SOC Console.
2. To understand Windows Event Viewer collection for Security, System and Application channels.
3. To examine SQLite persistence for logs, alerts, notifications, endpoint inventory and configuration.
4. To study rule-based detection, severity classification and multi-event correlation.
5. To evaluate endpoint monitoring for software, browsers, updates, USB devices and hardware/drivers.
6. To analyse network and firewall monitoring components, including ports and packet-rate evidence.
7. To understand the Flask/Jinja2 web dashboard, analytics, alert management and report-generation features.
8. To review the automated endpoint/security-rule tests supplied with the project.
9. To document actual runtime database statistics, limitations and possible future enhancements.

3. ABOUT THE ORGANIZATION

The internship reference material identifies the Defence Research and Development Organisation (DRDO) as the associated organization. DRDO is the research and development wing of the Ministry of Defence, Government of India, working on indigenous defence technologies and systems. Its activities cover a wide range of scientific and engineering areas relevant to national defence.

For this report, organization-specific claims are kept at the level supported by the supplied internship reference. The technical work documented here is the Windows Endpoint SIEM and SOC Console project supplied for analysis. The project itself is a local Windows security-monitoring application and should not be represented as a production DRDO infrastructure or as evidence collected from an organizational production network.

4. ROLE AT THE ORGANIZATION

During the internship, the project work involved the development and evaluation of a Windows endpoint monitoring and SIEM application. The role required understanding Windows security events, examining Python modules, validating database records, reviewing detection and correlation logic, and documenting the resulting architecture and behaviour.

4.1 Responsibilities

- Studied Windows Event Viewer channels and security-relevant Event IDs used by the collector.
- Reviewed the Flask application, database layer, collectors, monitors, detection engine and correlation engine.
- Analysed the SQLite database and verified stored runtime counts for logs, alerts and endpoint records.
- Reviewed rule-based detection for failed logins, audit-log clearing, service installation, time changes and other events.
- Evaluated software, browser, Windows update, USB and hardware/driver monitoring modules.
- Reviewed packet-rate, listening-port and Windows Firewall monitoring components.
- Examined alert severity, recommendations and MITRE ATT&CK-oriented enrichment.
- Reviewed the optional remote event-ingestion mechanism and its security requirements.
- Reviewed the supplied automated endpoint-rule test suite and documented its coverage.
- Prepared technical documentation describing architecture, implementation, observations, limitations and future enhancements.

Note: Runtime database values are reported only where they were directly available in the supplied SQLite database. Source-code support is not treated as proof that a feature was demonstrated in the stored runtime snapshot.

5. LEARNINGS AND EXPERIENCES

The internship provided practical exposure to Windows endpoint security, SIEM design, event collection, rule-based threat detection, correlation, endpoint baselining, SQLite database design and Flask web application development. The activities also strengthened the ability to distinguish between implemented functionality and functionality actually demonstrated by stored runtime data.

5.1 Project Overview and Architecture

The project follows a collection–storage–detection–correlation–presentation pipeline. Windows Event Viewer and Windows endpoint interfaces provide raw information. Specialized collectors and monitors convert this information into structured records. SQLite stores the records, the detection engine applies event rules and thresholds, and the correlation engine combines related findings. The Flask application then exposes dashboards, logs, alerts, analytics and reporting functions.

Layer	Main components	Function
Collection	collector.py; endpoint and system monitors	Collect Windows events and endpoint state.
Storage	database.py; data/siem.db	Persist logs, alerts, inventory and settings.
Detection	detector.py; severity_policy.py	Apply event rules, thresholds and severity.
Correlation	correlation_engine.py	Combine related findings.
Enrichment	recommendations.py	Explain findings and provide investigation guidance.
Presentation	app.py; templates/; static/	Dashboard, analytics, alerts and reports.
Remote ingestion	remote_collector.py; ingestion API	Optional event forwarding to a central instance.

5.1.1 Technology Stack

Technology	Use in project
Python	Core application and monitoring logic.
Flask	Web application and API layer.
SQLite	Local persistence.
Jinja2 / HTML / CSS / JavaScript	Server-rendered web interface.
Chart.js	Dashboard charts and visualizations.
Windows APIs / PowerShell	Event, update, system and security-state integration.
Windows Registry / PnP	Software, USB and hardware/driver information.
ReportLab / CSV	PDF and CSV report generation.

5.1.2 Major Project Files

File	Purpose
app.py	Flask application, routes, APIs and integration.
collector.py	Windows Event Viewer collection.
database.py	SQLite persistence and event fingerprinting.
detector.py	Rule-based event detection.
correlation_engine.py	Multi-event correlation.
severity_policy.py	Severity classification and explanations.
endpoint_monitor.py	Endpoint scan coordinator.
software_monitor.py	Installed software inventory and baseline.
browser_monitor.py	Browser version checks.
update_monitor.py	Windows update monitoring.
usb_monitor.py	USB inventory and first-seen monitoring.
hardware_monitor.py	Hardware/driver inventory and changes.
packet_monitor.py	Network packet-rate monitoring.

port_monitor.py	Listening-port monitoring.
firewall_monitor.py	Windows Firewall monitoring.
report_generator.py	CSV/PDF report generation.

5.1.3 High-Level Architecture

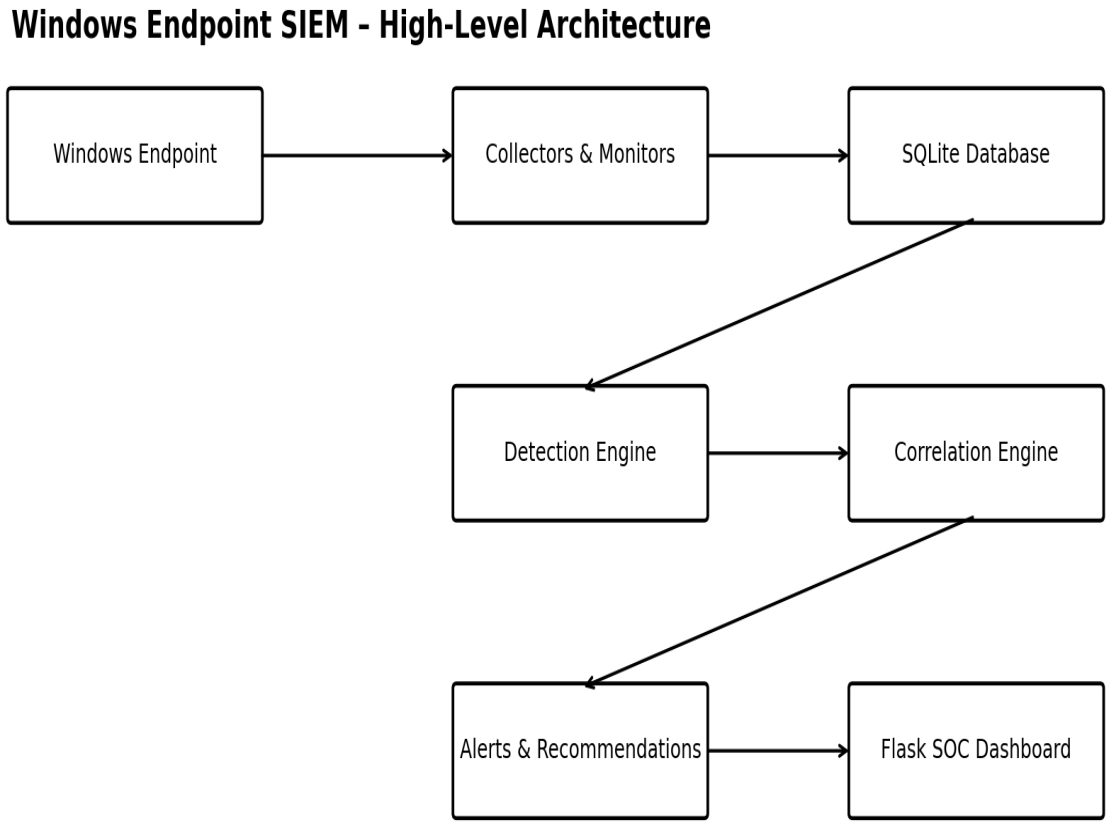


Figure 1: High-level architecture of the Windows Endpoint SIEM

The architecture shows the flow from a Windows endpoint through collectors and monitors into SQLite storage. The detection engine evaluates stored evidence, the correlation engine combines related conditions, and alerts/recommendations are presented through the Flask SOC dashboard.

5.2 WINDOWS EVENT COLLECTION

5.2.1 Objective

To understand how the project collects selected Windows Event Viewer records, normalizes important fields and stores them for investigation through the SIEM console.

5.2.2 Collection Behaviour

The collector supports Windows Security, System and Application channels and maps selected Event IDs to security meanings. The project records information such as event ID, channel, timestamp, computer, event type and message. Collection state is maintained per channel so that records can be processed incrementally.

5.2.3 Important Event Categories

Event ID	Observed / intended significance
7045	New service installed; security relevance depends on context.
6008	Unexpected shutdown.
1000	Application crash.
1001	Application hang/error-related event.
7031 / 7034	Service crash events.
219	Driver/device configuration event.
6005 / 6006	System startup and shutdown events.

5.2.4 Runtime Log Distribution

Channel	Records
System	500
Application	311
Security	0
Total	811

5.2.5 Most Frequent Event IDs

Event ID	Count	Interpretation
7045	412	New Service Installed
1001	245	Application Hang
1000	62	Application Crash
7031	27	Service Crash
219	20	Driver/Device Configuration
7034	19	Service Crash
6005	12	System Startup
6006	10	System Shutdown
6008	2	Unexpected Shutdown
1026	1	.NET Runtime Exception
20003	1	Driver Installation

The database is dominated by System and Application activity. Although the source code includes Security-channel rules for authentication and account-related detections, no Security-channel records are present in the supplied database. This distinction is important when reporting demonstrated results.

5.2.6 Event Collection Analysis

The stored log records demonstrate that the collector and database layer are operating on structured Windows events. Event ID 7045 is the most frequent event in the supplied snapshot, followed by Event IDs 1001 and 1000. These records represent service installation, application-hang and application-crash activity respectively.

The collector is designed to preserve original Event Viewer timestamps and channel information. This makes the stored records useful for later filtering, correlation and investigation. The project does not treat every event as malicious; rather, selected event types are passed to rule logic where their context determines whether a finding should be generated.

5.2.7 Practical Considerations

- Security Event log access works best when the console is run with appropriate Windows privileges.
- Event Viewer history depends on the source machine and its configured audit policies.
- Missing or unavailable Windows APIs should remain unavailable/unknown rather than being replaced by fabricated values.
- For a production deployment, collection reliability, retention and centralized storage would require additional engineering.

5.2.8 Key Learnings

- Understood the relationship between Windows Event Viewer and a SIEM collector.
- Learned how Event IDs and channels provide structured security evidence.
- Understood why per-channel collection state is useful for incremental monitoring.
- Learned to separate available source-code capabilities from observed runtime records.
- Observed how raw Windows events become searchable records in a database-backed security console.

5.2.9 Outcome

The event-collection component provides the foundation for the rest of the SIEM workflow. Once events are stored with consistent fields, the detection engine can apply explicit rules and the dashboard can present the records in a form suitable for investigation.

5.3 DETECTION AND CORRELATION ENGINE

5.3.1 Objective

To study how the project converts raw events and endpoint observations into security findings using explicit rules, thresholds, severity policy and multi-event correlation.

5.3.2 Detection Logic

The detector is rule-based rather than AI/ML-based. It evaluates conditions such as repeated failed logins, password spraying, administrator password activity, audit-log clearing, new services, system time changes and packet-rate thresholds. The severity policy centralizes the classification of findings into INFO, LOW, MEDIUM, HIGH and CRITICAL levels.

5.3.3 Major Detection Rules

Condition / Event	Project behaviour
Failed-login thresholds	Repeated failures can escalate into authentication findings and brute-force conditions.
Administrator password activity	Event IDs 4723/4724 can be evaluated in administrator context.
1102 – Audit log cleared	Critical security condition.
6008 – Unexpected shutdown	Security-relevant finding.
4697/7045 – New service	Severity depends on service/path context.
4616 – System time changed	Backdating thresholds determine severity.
Packet-rate threshold	Configured warning and critical thresholds classify packet-rate observations.

5.3.4 CORRELATION RULES

Rule / condition	Output
Clock backdating + audit-log clearing	CLOCK_CHANGE_AND_LOG_TAMPERING
Unregistered software + new service	SOFTWARE_SERVICE_PERSISTENCE
First-seen USB + unregistered software	USB_RELATED_SOFTWARE_INSTALL
Firewall disabled + Defender disabled	DEFENSES_IMPAIRED

5.3.5 Severity and Alert Handling

The supplied runtime database contains 724 alerts: 2 CRITICAL, 3 HIGH, 197 MEDIUM and 522 LOW. The project also contains 202 notification records. Alert records can carry severity, status, assignee/notes and supporting evidence, while the recommendations module provides investigation guidance and security context.

5.3.6 Key Learnings

- Rule-based detection is transparent and explainable but depends on carefully selected conditions and thresholds.
- Correlation can raise the significance of individually ambiguous observations.
- Severity communicates investigation priority and does not automatically prove compromise.
- Security findings should be interpreted together with the evidence and limitations shown by the application.

5.3.7 Automated Rule Validation

The supplied project includes tests/test_endpoint_rules.py. The test module covers time-change thresholds, software allowlist and baseline behaviour, browser UNKNOWN handling, port inventory explanations, administrator password detection, packet-rate thresholds and hardware/driver changes.

Test area	Verified behaviour
Time integrity	20-second, 60-second and five-minute backdating thresholds map to different severity levels; future time changes are distinguished.
Software baseline	Empty allowlist baselines known software and later identifies new software.
Browser status	Unknown minimum version is returned as UNKNOWN rather than automatically treating the version as outdated.
Port inventory	Known ports receive explanatory purposes; unknown listeners receive an uncertainty note.
Administrator password	Event 4724 for Administrator maps to ADMINISTRATOR_PASSWORD_CHANGED / CRITICAL.
Packet rate	Configured thresholds classify PASS, HIGH and CRITICAL rates.
Hardware changes	Driver changes and missing baseline devices are detected after a baseline exists.

The supplied analysis reports nine endpoint/security-rule tests passing. This provides basic automated validation of individual rule functions; broader integration, deployment and security testing would still be required for production use.

5.4 ENDPOINT MONITORING

5.4.1 Objective

To understand how the project extends event monitoring with endpoint inventories and security-state baselines.

5.4.2 Software Monitoring

The software monitor reads installed-software information from the Windows Registry. It can operate with an approved-software configuration or with baseline mode. With an empty allowlist, the first scan establishes a baseline and later appearances of previously unseen software can be identified as NEW_SINCE_BASELINE. The supplied database contains 87 software inventory records and 12 software-change records.

5.4.3 Browser Monitoring

The browser monitor checks supported browsers including Chrome, Edge, Firefox and Brave against configured minimum versions and can identify downgrades. The supplied database contains two browser inventory records. The stored browser versions are UNKNOWN while their status is OUTDATED, so this result should be treated as a data/code-version consistency issue rather than as definitive proof of an outdated browser.

5.4.4 Windows Update Monitoring

The update monitor uses Windows hotfix information and checks whether a recent installed update can be established. The database contains 84 system-update records. An UNKNOWN update result should not be presented as definitive proof that an endpoint is unpatched.

5.4.5 USB Monitoring

The USB monitor uses Windows Plug and Play information to maintain a device inventory and identify first-seen devices. The supplied database contains 4 USB devices and 84 USB events. USB evidence can also participate in the USB_RELATED_SOFTWARE_INSTALL correlation rule when combined with unregistered software evidence.

5.4.6 Hardware and Driver Monitoring

The hardware monitor uses signed-driver and device information to maintain a baseline. It can detect driver version/date/signature changes and disappearance of a previously observed device or driver. The supplied database contains 171 hardware inventory records and 1 hardware-change record. Driver/device disappearance is evidence requiring validation because unplugging or legitimate uninstallation can produce the same observation.

5.4.7 Security Settings

Endpoint scans include firewall, RDP and UAC setting snapshots. The first scan establishes a baseline and later changes can be recorded. The supplied database contains 3 system-settings records but no system-setting change rows.

5.4.8 Endpoint Snapshot

The runtime database contains 21 endpoint snapshots. These snapshots provide a compact representation of endpoint state and complement the more detailed inventory tables.

5.4.9 Key Learnings

- Endpoint baselining provides context for changes that cannot be interpreted from isolated events.
- Inventory monitoring must distinguish a real security change from normal administrative activity.
- UNKNOWN values should remain explicit rather than being converted into unsupported positive or negative claims.
- USB, hardware and software evidence can become more meaningful when correlated with other observations.

5.4.10 Endpoint Monitoring Outcome

The endpoint-monitoring layer expands the SIEM beyond Event Viewer records. It gives the analyst visibility into installed software, browsers, operating-system updates, USB devices, hardware/drivers and selected security settings. This broader context supports investigation and helps explain whether an event is isolated or associated with a change in endpoint state.

5.5 A QUANTITATIVE EVALUATION OF THE DEVELOPED SIEM

5.5.1 Evaluation Approach

The reference internship report uses a five-point subjective quality scale. For this project, the more defensible approach is to report measurable implementation evidence instead of assigning unsupported quality scores. The following values come from the supplied source archive and populated SQLite database.

Metric	Evidence from supplied project
Stored logs	811
Stored alerts	724
Notifications	202
Software inventory	87
System updates	84
USB events	84
Hardware inventory	171
Endpoint snapshots	21
Network-flow baselines	1
Automated rule tests	9 tests; supplied analysis reports all 9 passing.
Detection model	Rule-based; no AI/ML threat classifier in supplied implementation.
Reporting	CSV and PDF export functions.

5.5.2 ACTUAL DATABASE SNAPSHOT

Table	Rows
logs	811
alerts	724
notifications	202
software_inventory	87
software_changes	12
system_updates	84
browser_inventory	2
usb_devices	4
usb_events	84
hardware_inventory	171
hardware_changes	1
endpoint_snapshots	21
network_flow_baselines	1
system_settings	3
console_users	0

5.5.3 Alert Severity Distribution

Severity	Alerts
CRITICAL	2
HIGH	3
MEDIUM	197
LOW	522

The stored database demonstrates that the project has processed actual structured endpoint data. The evidence is strongest for System and Application event collection and for endpoint inventories. Security-channel authentication detections exist in code but are not demonstrated by the supplied database.

5.6 SOC DASHBOARD AND WEB APPLICATION

5.6.1 Interface

The web interface is implemented using Flask, Jinja2 templates, HTML, CSS and JavaScript. Chart.js is used for visualizations. The application is not a React or Node.js frontend.

5.6.2 Main Pages

Page / feature	Purpose
Dashboard	Overview of logs, alerts, severity and monitoring information.
Logs	Search and filter collected event records and expand explanations.
Alerts	View findings, severity, evidence, recommendations and status.
Analytics	Display trends, distributions and security statistics.
Systems	Show system/host-oriented information.
Active Users	Display current Windows sessions and related information.
Ports	Display TCP/UDP listeners and process information where available.
Firewall	Show Windows Defender Firewall state and related evidence.
Threat Intelligence	Present project-generated security context and recommendations.
Reports	Export logs and alerts to CSV/PDF.
Settings / Users	Provide available configuration and optional local authentication flows.

5.6.3 Alert Investigation

The console is designed to turn raw technical events into an investigation workflow. An analyst can move from the dashboard to logs, inspect an alert, review its severity and evidence, and then use recommendation/context fields to understand what should be checked next. This is a rule-based analyst-assistance workflow rather than automated incident response.

5.6.4 Analytics and Reporting

The project includes dashboard charts and report-generation functions. Report exports are available in CSV and PDF formats. The reporting layer is useful for documenting findings, sharing a snapshot of collected evidence and supporting internship demonstrations.

5.6.5 Remote Collection

The optional remote_collector.py component can forward selected Windows event records to a central instance through the ingestion API. The README recommends strong API keys, HTTPS/reverse-proxy protection, firewall restrictions and secure deployment before using this capability on a real network. The default project remains a local single-endpoint console.

5.6.6 SECURITY CONSIDERATIONS

- Do not expose the default no-login local configuration to a shared network.
- Protect the SQLite database because it can contain usernames, host information and security events.
- Protect and rotate remote-ingestion API keys when used.
- Use HTTPS and secure session configuration for non-local deployments.
- Database ACL protection is not encryption; encryption at rest should use an appropriate Windows mechanism.
- Use suitable retention policies and protect database backups.

5.6.7 Working of the System

1. The local Windows endpoint generates Event Viewer and endpoint-state information.
2. Collectors and endpoint monitors gather selected evidence.
3. database.py stores normalized records in SQLite and supports event fingerprinting.
4. detector.py evaluates rule-based conditions and severity_policy.py classifies findings.
5. correlation_engine.py combines related findings into higher-level alerts.
6. recommendations.py adds investigation guidance and security context.
7. app.py exposes the results through Flask/Jinja2 pages and APIs.
8. The dashboard and analytics pages provide visual investigation views.
9. Reports can be exported to CSV/PDF for documentation.

5.6.8 LIMITATIONS

- The system is an educational local SIEM and is not a full EDR or managed enterprise SIEM replacement.
- The supplied database contains no Security-channel logs, so authentication detections are not demonstrated in the stored snapshot.
- Browser and update checks can return UNKNOWN values and should be interpreted carefully.
- Packet monitoring uses adapter counters and nearby TCP connection snapshots; it cannot prove process-level attribution of a packet spike.
- The firewall monitor is primarily read-only and does not automatically block traffic.
- Configuration fields for integrations such as SSO/MFA/Syslog/webhooks should not be described as complete integrations without additional implementation.
- SQLite is suitable for the supplied local prototype but larger deployments would require stronger storage and access controls.
- The packaged ZIP contains a large virtual environment and cached files; clean distribution should separate dependencies from source code.

5.6.9 FUTURE ENHANCEMENTS

1. Add a validated machine-learning anomaly-detection layer while retaining explainable rules.
2. Validate Security-channel authentication detections using representative test data.
3. Improve UNKNOWN handling for browser and update checks.
4. Add genuine Syslog, webhook and external threat-intelligence integrations.
5. Implement production-grade MFA/SSO where required.
6. Add richer process telemetry and process-tree analysis.
7. Add stronger multi-endpoint identity, TLS enforcement and health monitoring.
8. Use PostgreSQL or another production database for larger deployments.
9. Expand integration and security testing.
10. Add controlled automated response actions with authorization and audit logging.

5.6.10 OBSERVATIONS AND OUTCOMES

The project demonstrates a complete educational SIEM workflow from endpoint collection through persistence, detection, correlation, enrichment, dashboard visualization and reporting. The populated database confirms that the application has processed hundreds of structured events and endpoint records. The supplied test module also covers time integrity, software baselining, browser handling, port inventory, administrator password activity, packet thresholds and hardware changes.

The main practical outcome of the internship was a clearer understanding of how security-monitoring components fit together and why evidence quality, baselines, context and limitations matter when interpreting security alerts.

Practical Skills Developed

- Python and Flask application development.
- SQLite schema inspection and database-backed security monitoring.
- Windows Event Viewer and Windows endpoint integration concepts.
- Rule-based detection and severity classification.
- Multi-event correlation and alert enrichment.
- Endpoint inventory and baseline analysis.
- SOC dashboard, analytics and reporting concepts.
- Automated testing and technical documentation.

5.6.11 KEY LEARNINGS AND DEMONSTRATION SEQUENCE

The supplied README provides a practical demonstration sequence for the project. A suitable presentation can follow the same order so that each major component is shown with its corresponding evidence.

1. Open Dashboard and show local counts and charts.
2. Open Logs and point out the Log Type column and an event explanation.
3. Open Alerts, use the severity guide and explain the evidence for one alert.
4. Open Security and run Run Security Scan to show software, updates, browsers, USB, hardware and driver data.
5. Open Firewall Monitor to show current profiles, version/audit status and collected firewall evidence.
6. Open Analytics to demonstrate event and alert distributions.
7. Open Reports to demonstrate CSV/PDF export.
8. Show the automated endpoint-rule test output and explain the covered rule categories.

For a submission demonstration, the most important principle is to show the actual behaviour of the supplied implementation and to clearly distinguish between stored runtime evidence, source-code capabilities and proposed future enhancements.

6. CONCLUSION

The Windows Endpoint SIEM and SOC Console is a functional Windows-focused security monitoring application that combines Windows Event Viewer collection, endpoint inventory, rule-based detection, alert correlation, severity classification, recommendations, analytics and reporting within a Flask web application.

The supplied runtime database demonstrates that the system has collected and processed real structured records: 811 logs, 724 alerts, 202 notifications, 87 software inventory records, 84 system-update records, 84 USB events, 171 hardware inventory records and 21 endpoint snapshots.

The analysis also identifies important limits. The current detection engine is rule-based rather than AI/ML-based. The supplied database contains no Security-channel records even though Security-channel detection is implemented in source code. Browser and update checks can produce UNKNOWN values, and packet-rate evidence cannot by itself prove process-level attribution.

Overall, the project provided practical exposure to cybersecurity monitoring, Windows internals, Python development, database design, web application development, security-rule analysis, testing, technical documentation and incident-oriented thinking. It establishes a practical educational foundation for future work in centralized collection, richer telemetry, stronger authentication, external integrations and production deployment hardening.

7. APPENDIX

Appendix A: Major Project Files

File / folder	Purpose
app.py	Flask application, routes, APIs and integration
collector.py	Windows Event Viewer collection
remote_collector.py	Optional outbound remote event forwarder
endpoint_monitor.py	Endpoint scan coordinator
detector.py	Threat detection rules
correlation_engine.py	Multi-alert correlation
severity_policy.py	Severity and event explanations
recommendations.py	Alert explanations and response guidance
database.py	SQLite schema, inserts and queries
hardware_monitor.py	Hardware and driver baseline monitoring
software_monitor.py	Installed software inventory and policy
browser_monitor.py	Browser version and downgrade checks
update_monitor.py	Windows update monitoring
usb_monitor.py	USB inventory and first-seen device alerts
packet_monitor.py	Packet-rate and TCP evidence
port_monitor.py	Local listener inventory
firewall_monitor.py	Defender Firewall posture
report_generator.py	CSV and PDF report generation
tests/test_endpoint_rules.py	Automated endpoint and detection tests
config/approved_software.json	Approved software configuration
data/siem.db	Populated runtime SQLite database

7. APPENDIX

Appendix B: Actual Database Snapshot

Metric	Value
Total logs	811
Total alerts	724
Critical alerts	2
High alerts	3
Medium alerts	197
Low alerts	522
Notifications	202
Software inventory	87
Software changes	12
System updates	84
USB devices	4
USB events	84
Hardware inventory	171
Hardware changes	1
Endpoint snapshots	21
Network baselines	1
System settings	3
Console users	0

Appendix C: Important Detection Rules

Rule / condition	Output
3+ failed logins	Multiple failed-login finding / escalation
8+ failed logins	Brute-force finding
Failures against multiple users from a source	Password-spray finding
4723/4724 + administrator context	Administrator password activity
1102	Audit log cleared – critical
4697/7045	New service installed; context affects severity

4616 + significant backwards time movement	System clock backdating
Clock backdating + audit log clearing	CLOCK_CHANGE_AND_LOG_TAMPERING
Unregistered software + new service	SOFTWARE_SERVICE_PERSISTENCE
First-seen USB + unregistered software	USB_RELATED_SOFTWARE_INSTALL
Firewall disabled + Defender disabled	DEFENSES_IMPAIRED

8. CITATIONS

1. Supplied Windows Endpoint SIEM project archive, including Python modules, Flask templates, configuration files, automated tests and populated SQLite database.
2. Supplied internship report PDF used as the formatting and organizational reference for this report.
3. Project runtime database: data/siem.db contained in the supplied project archive.
4. Supplied project README describing installation, monitoring scope, operational limits, demonstration sequence and deployment guidance.

9. REFERENCES

1. Python Software Foundation. Python documentation.
2. Pallets Projects. Flask documentation.
3. Microsoft. Windows Event Log and Windows security auditing documentation.
4. Microsoft. Windows Management Instrumentation and PowerShell documentation.
5. SQLite Documentation. SQLite database engine documentation.
6. Chart.js Documentation. JavaScript charting library documentation.
7. ReportLab Documentation. PDF generation library documentation.
8. MITRE ATT&CK. Enterprise ATT&CK knowledge base.
9. Supplied Windows Endpoint SIEM and SOC Console source archive and test suite.
10. Supplied internship report PDF used as the report-format reference.